INFOSYS × IISC × NVIDIA · AEGIS 2026

MemLedger

Provenance-governed memory for long-running AI agents

Team Prompt Stars: Banaj Mahajan, Ahan Phatak, Omkaar Nerurkar, Aditya Garg

Problem statement: Agent Memory Governance Evaluation

ORGANISED BY

Infosys® Navigate your next

HOSTED BY

Indian Institute of Science
भारतीय विज्ञान संस्थान

TECHNOLOGY PARTNER

NVIDIA

1 · PROBLEM STATEMENT

Agent Memory Governance Evaluation

> Agents that run for weeks store what they read and act on it later. Poisoned, untraceable, contradictory or leaked memory steers them without anyone noticing.

Security
Resist poisoned and tampered memory

Privacy
Keep users' and teams' memory apart

Safety
No harmful action from a bad memory

Reliability
Know each fact's origin; no made-up facts

50.3% of memory-poisoning test cases ended in a real attacker-steered action, across 24 agent configurations (MemSecBench [4], not our data).

Five ways an agent’s memory goes wrong

Problem	What it looks like	Pillar
Poisoning	A web page says a vendor’s account changed; weeks later the agent pays it	Security, Safety
Unknown origin	A claim repeated by one site looks as reliable as a paper	Reliability
Hallucination, conflicts	The agent stores a summary its sources do not support	Reliability
Leaks	Team A’s agent answers with team B’s figures	Privacy
Tampering	Someone edits stored records directly in the database	Security

Why today’s defences fall short

Poison reads like an ordinary fact, so injection filters miss it [7]

Plain retrieval returns whatever is most similar, planted notes included

The model alone decides; once it is fooled, nothing stops the action

3 · OUR SOLUTION

MemLedger: the agent proposes, the service decides

A memory layer between the agent and its storage. Four core ideas:

1 Provenance by the system

Each memory is linked to the sources that really support it, as checked by a fact-checker.

2 Trust can’t be inflated

As trusted as its weakest source; only independent origins add trust.

3 Targeted clean-up

Flag a bad source: everything derived from it is quarantined, the rest stays.

4 Hard guarantees in code

Signed registry for critical facts, risky actions gated, team isolation and hash checks.

3 · HOW WE TACKLE IT: ONE ATTACK, STEP BY STEP

“Acme’s bank account has changed to 9876”

The agent reads this line on a supplier’s web page. Two weeks later, a finance user asks it to pay Acme’s latest invoice.

PLAIN RETRIEVAL MEMORY

1 Stored like any other memory.

2 Recalled as relevant context for the payment.

3 Acted on. The agent pays account 9876. Nothing records why; memories built on the page stay.

WITH MEMLEDGER

1 Recorded as a low-trust web source, linked to the claim.

2 Rejected. Bank accounts are protected; the claim fails the signed registry and the page is flagged.

3 Paid safely. The payment tool reads Acme’s account from the registry itself.

4 Cleaned up. Everything derived from the page is quarantined.

3 · HOW WE TACKLE IT: EVERY WRITE AND EVERY READ

Eight steps, two lanes

A · WRITING A MEMORY



STORAGE · per-team vector + keyword index · provenance graph + change log · signed registry (Ed25519) · append-only hash log

B · USING A MEMORY



Access control in code runs under every step: login identity, team scope, labels, hash checks. Navy = the AI agent; white = the memory service; grey = outside the system.

4 · PERFORMANCE · TEST SPLIT, NEMOTRON NANO 9B V2

Attacks drop from one in two to one in 25

3.9%

of attacks succeed against full MemLedger, vs **48.9%** on plain memory

0

hijacked actions (of 838) and cross-team leaks (of 2,299 runs)

5.6%

of harmless writes rejected, at +21.1 ms per write. Our ≤ 5% target: **not met**

Evaluation run of 27 Oct 2026 · 2,299 episodes per version · 95% intervals in the repository

4 · PERFORMANCE: ONE LAYER AT A TIME

Each layer removes a class of attack

Version	Attack success	Hijacked	Leaks	Tamper caught	Harmless rejected	Task accuracy	ms / write
Baseline	**48.9%**	35.6%	14.4%	0/72	0.0%	86.1%	–
+ Provenance	32.1%	32.7%	4.0%	72/72	0.0%	86.1%	+0.2
+ Write checks	12.8%	6.9%	3.8%	72/72	5.5%	84.5%	+20.4
Full MemLedger	**3.9%**	**0/838**	**0/2299**	72/72	5.6%	84.0%	+21.1
Full, no registry	4.0%	0/838	0.7%	72/72	5.6%	84.0%	+19.3

Test split, 2,299 episodes per version, Nemotron Nano 9B v2 on vLLM (NVIDIA L40). Hijacked = high-risk calls run with attacker values (of 838). Latency from a 1-worker run. With Qwen3-8B as the agent, full MemLedger gives nearly the same result (4.2%).

4 · PERFORMANCE: ESTABLISHED BENCHMARKS

It holds on benchmarks we never tuned on

Benchmark (test episodes)	Baseline	Full	Harmless blocked	Stopped most often by
AgentDojo (230) · test only	43.0%	**0.0%**	–	input screen
InjecAgent (600) · test only	0.7%	**0.0%**	0.0%	input screen
MemShield-bench (179)	93.9%	10.2%	3.1%	memshield
Our scenarios (740)	81.8%	**4.0%**	2.0%	read filter, hash, registry
BIPIA (300)	13.3%	**10.0%**	8.7%	input screen · weak spot
MemEvoBench (166)	21.7%	**13.3%**	12.7%	memshield · weak spot

Attack success over attack episodes only (AgentDojo 230, BIPIA 150, InjecAgent 300, MemEvoBench 83, MemShield-bench 49, scenarios 644). AgentDojo: 0 of 23 injection tasks succeed on full. Harmless blocked = clean inputs or memories wrongly stopped.

What it costs, and where it still falls short

Cost in normal use

+21 ms per memory write, **+0.7 ms** per read

2.1 points task accuracy lost (86.1% → 84.0%)

5.6% harmless writes rejected (target ≤ 5%: not met)

0 of 66 legitimate payments held or refused

≈ $19 GPU spend for all runs together (~18 L40 hours)

Weak spots → next step

PoisonedRAG 43% still succeed → a better hallucination rule than “weakest fact decides”

MemEvoBench 13% → find the missing check, retrain memshield

BIPIA 10%, 8.7% clean emails blocked → use the screen’s “untrusted” middle band

IN ONE SENTENCE

The model proposes, the code decides: a fooled agent still cannot pay the attacker or leak another team’s data.

3.9%
attack success (vs 48.9%)

0
hijacks, leaks

72/72
database edits caught

+21 ms
per write; 5.6% harmless rejected

Thank you. Questions? · Team Prompt Stars